

Contents

1	Introduction	2
1.1	Background	2
1.2	Problem Statement	3
1.3	Objective	3
1.4	Scope	4
2	Methodology	5
2.1	Ideal Stereo Setup & Theory	5
2.2	Image Capture	6
2.3	Camera Calibration	7
2.3.1	Checkerboard Calibration	7
2.3.2	Intrinsic Calibration	8
2.3.3	Stereo Calibration	8
2.4	Stereo Geometry Estimation	10
2.5	SIFT Feature matching	10
2.6	Image Rectification	11
2.6.1	Rectification Process	11
2.6.2	Explanation of Code	12
2.6.3	Rectification Results	12
2.6.4	Disparity Map Generation	13
2.7	Disparity Map Generation	13
2.7.1	Block Matching	14
2.7.2	WLS Filtering	14
2.7.3	Disparity Map Visualization	15
2.7.4	Disparity Map Results	15
2.8	Next Steps	15
2.9	Triangulation & 3D Reconstruction	16
2.9.1	Reprojection of Image Points to 3D Coordinates	17
2.9.2	3D Point Cloud Visualization	17
2.9.3	3D Reconstruction Results	18
3	Discussion	18
3.1	Performance Evaluation	19
3.2	Challenges and Limitations	19
3.3	Potential Improvements	20
3.4	Applications	21
4	Conclusion	21
4.1	Impact of the Work	22
4.2	Future Work	22

Stereo

Osama Fawad, Muhammad Haseeb khan, Sajeed Hussian

May 2025

Abstract

This project presents the implementation of a stereo vision pipeline using two parallel webcams to generate a dense 3D reconstruction of a scene using OpenCV. The pipeline consists of several stages, including image capture, camera calibration, stereo geometry estimation, image rectification, disparity map generation, feature matching (optional), and triangulation to create a dense 3D point cloud. The system is designed to use low-cost, widely available hardware, specifically two webcams placed in a fixed parallel configuration with a baseline of 32 cm.

The calibration process uses a checkerboard grid to compute intrinsic and stereo calibration parameters. A robust stereo geometry estimation technique is applied to compute the relative pose between the two cameras. Image rectification aligns the two images to allow for accurate disparity map generation using the StereoBM method with a WLS filter for enhanced depth perception. Feature matching using the SIFT algorithm is implemented as an optional step for sparse matching, aiding in pose estimation and essential matrix computation.

The final step involves triangulation, where disparity values are used to compute 3D coordinates, which are then visualized as a point cloud. The results demonstrate the effectiveness of the pipeline in generating accurate 3D reconstructions, making it a valuable tool for applications in robotics, computer vision, and augmented reality, where depth perception is critical. This implementation leverages OpenCV's powerful computer vision tools to provide an accessible and cost-effective method for dense 3D reconstruction.

1 Introduction

1.1 Background

Stereo vision is a computer vision technique that uses two or more cameras placed at different positions to simulate depth perception. By comparing two images captured from slightly different viewpoints, it is possible to infer the 3D structure of the scene. Stereo vision systems are widely used in fields such as robotics, autonomous vehicles, augmented reality, and 3D modeling. These systems typically use the disparity between corresponding points in the stereo

pair of images to estimate the depth, which is then used to create a 3D model of the scene.

This technique mimics the way human vision works, where the brain combines images from both eyes to perceive depth. For accurate depth estimation, key factors like camera calibration, image rectification, and disparity map generation are crucial. Calibration helps determine the intrinsic and extrinsic parameters of the cameras, while image rectification ensures that the corresponding points in both images lie on the same horizontal line, simplifying the disparity calculation. The disparity map then provides depth information, which can be used to generate a 3D point cloud.

In this project, stereo vision is implemented using two parallel webcams and OpenCV to generate a dense 3D reconstruction of a scene. The primary goal is to develop a cost-effective method for real-time 3D reconstruction that can be applied in various fields like robotics and augmented reality, where accurate depth perception is essential.

1.2 Problem Statement

While stereo vision has been extensively studied and applied in various domains, many existing solutions require expensive and specialized hardware such as high-end cameras or stereo rigs. Moreover, the complexity of real-time 3D reconstruction often makes these systems inaccessible for casual or low-budget applications. A common challenge faced in stereo vision systems is the calibration and alignment of the cameras, as any misalignment can lead to significant errors in depth estimation and 3D reconstruction.

Additionally, generating a dense 3D reconstruction from stereo images requires accurate image rectification and disparity map generation. These processes can be computationally intensive, especially when working with real-time data. This project aims to address these challenges by utilizing standard webcams and optimizing the pipeline to provide a balance between performance and accuracy.

The goal is to implement a stereo vision pipeline that is both affordable and effective for generating 3D reconstructions using OpenCV. The system will provide accurate depth information and create a dense 3D point cloud, making it suitable for various applications, from robotics to virtual environments.

1.3 Objective

The main objective of this project is to implement a full stereo vision pipeline that generates a dense 3D point cloud of a scene using two parallel webcams. The system will be implemented using OpenCV, which provides powerful computer vision tools. The pipeline consists of the following steps:

- **Image Capture:** Capturing synchronized stereo images from two webcams.

- **Camera Calibration:** Determining the intrinsic and extrinsic parameters of the cameras using a checkerboard grid.
- **Stereo Geometry Estimation:** Estimating the relative pose (rotation and translation) between the two cameras.
- **Image Rectification:** Aligning the stereo images to simplify disparity map generation.
- **Disparity Map Generation:** Calculating depth information from the aligned stereo images.
- **Feature Matching (Optional):** Using SIFT (Scale-Invariant Feature Transform) for sparse feature matching to improve pose estimation.
- **Triangulation and 3D Reconstruction:** Generating a dense 3D point cloud from the disparity map.

The goal is to develop a real-time stereo vision system that can produce accurate and detailed 3D models from standard webcams, enabling its application in low-budget settings.

1.4 Scope

The scope of this project focuses on building a cost-effective stereo vision pipeline that utilizes two parallel webcams to capture stereo images and reconstruct a 3D scene. The primary software tool used is OpenCV, which provides functions for camera calibration, disparity map generation, and 3D point cloud visualization. The system is designed to be accessible, requiring only standard webcams and basic hardware.

This project includes the following:

- A stereo camera setup consisting of two parallel webcams with twist of using `cv2.stereocalibrate` which allows auto baseline detection making it autonomous in this sense.
- Camera calibration using a checkerboard grid to compute intrinsic and extrinsic parameters.
- Stereo geometry estimation to compute the relative pose between the cameras.
- Image rectification to align the stereo images for accurate disparity map generation.
- Disparity map generation using block matching and filtering techniques.
- Optional feature matching using SIFT for pose estimation.
- 3D reconstruction using triangulation based on the disparity map.

While this project aims to create real-time 3D reconstructions, it prioritizes simplicity and accuracy over high performance. The resulting system will produce dense 3D point clouds that are visualized using matplotlib.



Figure 1: Test Image Used for Stereo Vision Pipeline (Left Image)

The system will be evaluated based on the accuracy of the 3D reconstruction and the ease of use, with a focus on achieving high-quality depth maps using standard webcam hardware.

2 Methodology

This section describes the detailed methodology followed in the project for stereo vision-based 3D reconstruction. Each step in the stereo vision pipeline is explained, from capturing stereo images to generating a 3D point cloud.

2.1 Ideal Stereo Setup & Theory

Stereo vision relies on the concept of depth perception, achieved by comparing two images captured from different viewpoints. The following setup was used for this project:

- Two parallel webcams are placed at a fixed baseline of 32 cm, ensuring that both cameras capture images at the same height and angle.
- The resolution of both cameras is set to 1920x1080 pixels, allowing for a detailed capture of the scene.
- The cameras are calibrated to ensure their intrinsic and extrinsic parameters are accurate for disparity map generation and 3D reconstruction.

The setup is crucial for ensuring accuracy in the disparity map generation and the subsequent 3D point cloud reconstruction. Below is the baseline photo of the camera setup used in the project.

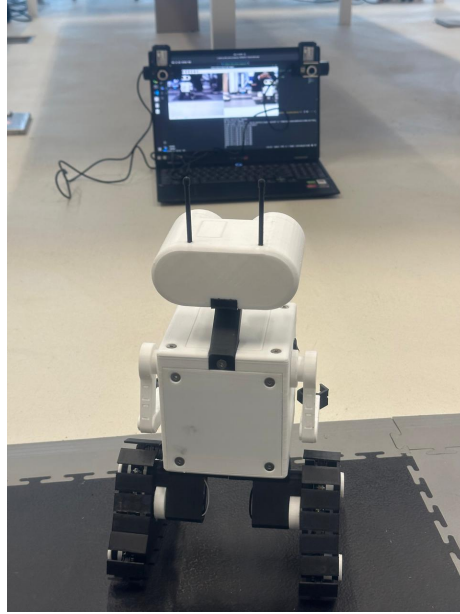


Figure 2: Stereo Camera Setup with a 32 cm Baseline.

2.2 Image Capture

Image capture is the first step in the stereo vision pipeline. To ensure synchronized image capture from the two webcams, the following process is followed:

- The left and right cameras are accessed using the `cv2.VideoCapture()` function.
- A loop is created to continuously capture frames from both cameras at the same time, ensuring synchronization.
- The captured frames are stored in separate directories for the left and right images for easy processing during calibration and disparity map generation.

The code for capturing synchronized stereo images from the webcams is as follows:

```
# Initialize the cameras
left_camera = cv2.VideoCapture(0) # Left camera
right_camera = cv2.VideoCapture(1) # Right camera
```

```

# Capture the frames from the cameras
ret1, left_frame = left_camera.read()
ret2, right_frame = right_camera.read()

# Check if frames are captured correctly
if ret1 and ret2:
    cv2.imwrite('calibration_photos/left/img_{0}.jpg'.format(frame_id), left_frame)
    cv2.imwrite('calibration_photos/right/img_{0}.jpg'.format(frame_id), right_frame)
else:
    print("Failed to capture images")

```

This code initializes the two cameras using `cv2.VideoCapture()` and captures images from both webcams. The captured frames are saved to the respective directories for further processing in later stages of the pipeline.

2.3 Camera Calibration

The next step in the pipeline is camera calibration, which is performed to determine the intrinsic and extrinsic parameters of the cameras. Intrinsic parameters refer to the internal camera properties such as focal length and optical center, while extrinsic parameters describe the position and orientation of the cameras relative to each other.

2.3.1 Checkerboard Calibration

To perform camera calibration, a checkerboard pattern is used. The pattern has 7x9 inner corners, which are detected in the left and right images. The function `cv2.findChessboardCorners()` is used to detect these corners, and `cv2.cornerSubPix()` refines the corner locations for more accuracy.

The code for corner detection and calibration is as follows:

```

# Load the image and convert to grayscale
image = cv2.imread(image_path)
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Find chessboard corners
ret, corners = cv2.findChessboardCorners(gray, (7, 9), None)

# Refine corner positions
if ret:
    cv2.cornerSubPix(gray, corners, (11, 11), (-1, -1), criteria)
    cv2.drawChessboardCorners(image, (7, 9), corners, ret)
    cv2.imshow('Calibration Corners', image)
    cv2.waitKey(0)

```

`cv2.findChessboardCorners()` detects the corners, while `cv2.cornerSubPix()` refines their positions for better accuracy. This step ensures precise corner detection, which is vital for accurate calibration.

2.3.2 Intrinsic Calibration

Intrinsic calibration involves determining the camera's internal parameters, including the focal length, optical center, and distortion coefficients. This is done using the function `cv2.calibrateCamera()`, which requires the 3D object points (the real-world coordinates of the corners) and the 2D image points (the detected corners in the images).

The code for intrinsic calibration is as follows:

```
# Object points (3D coordinates of the checkerboard corners)
object_points = np.zeros((num_corners, 3), np.float32)
object_points[:, :2] = np.mgrid[0:7, 0:9].T.reshape(-1, 2) # 7x9 grid

# Camera calibration
ret, camera_matrix, dist_coeffs, rvecs, tvecs = cv2.calibrateCamera(
    object_points, image_points, image_size, None, None)
```

This code computes the camera matrix (`camera_matrix`) and distortion coefficients (`dist_coeffs`) that describe the camera's internal properties. Calibration results also include the rotation vectors (`rvecs`) and translation vectors (`tvecs`), which are used in stereo calibration.

2.3.3 Stereo Calibration

Stereo calibration computes the relationship between the two cameras, i.e., the rotation (R) and translation (T) matrices that define the relative position and orientation of the cameras. The function `cv2.stereoCalibrate()` is used to calculate the stereo calibration parameters, which are essential for aligning the stereo images and generating an accurate disparity map.

The code for stereo calibration is as follows:

```
# Stereo calibration to compute relative pose between the two cameras
ret, camera_matrix1, dist_coeffs1, camera_matrix2, dist_coeffs2,
R, T, E, F = cv2.stereoCalibrate(object_points, image_points1, image_points2,
                                camera_matrix1, dist_coeffs1, camera_matrix2,
                                dist_coeffs2, image_size, None, None, None, None)
```

The stereo calibration results include:

- R - Rotation matrix: Describes the relative orientation between the two cameras.
- T - Translation vector: Describes the relative position of the cameras.
- E - Essential matrix: Encodes the geometric relationship between the two views.

- **F - Fundamental matrix:** Contains information about the point correspondences between the stereo images.

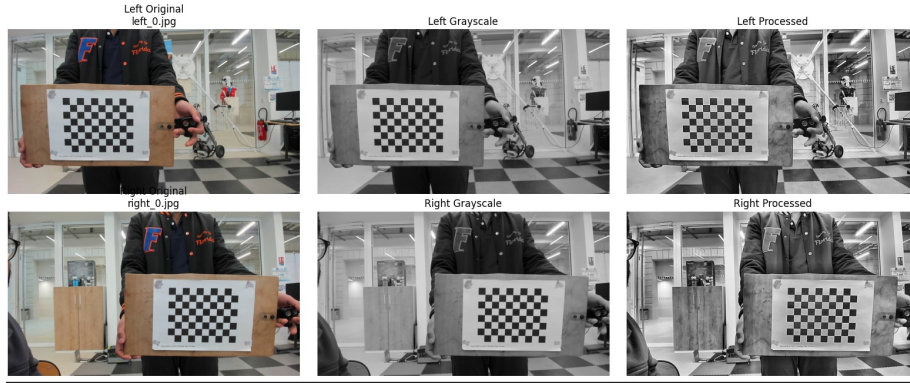


Figure 3: The oriiginal and processed images before calibration.

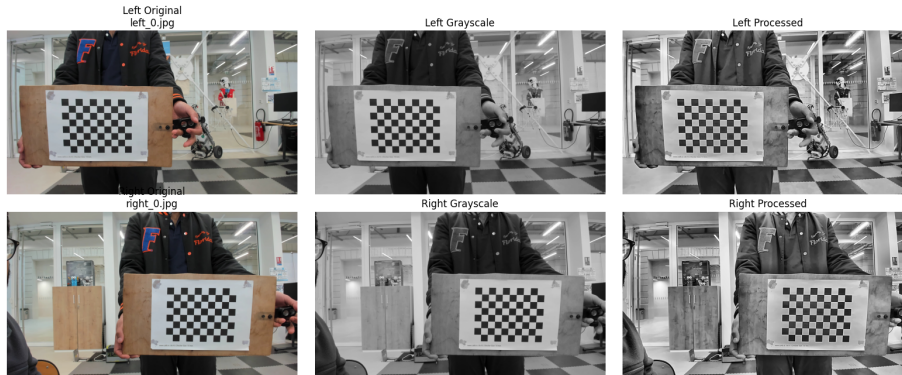


Figure 4: Calibration pre-processing.

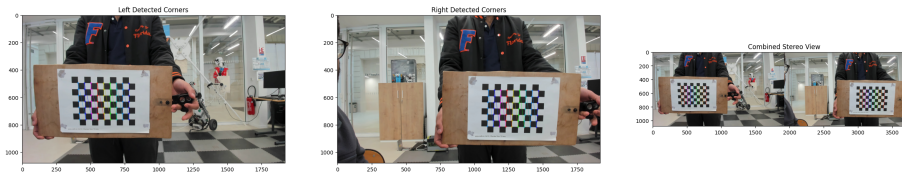


Figure 5: after Calibration corner detection.

2.4 Stereo Geometry Estimation

There are two different approaches tried in different ways. first is this one that we did for sparse reconstruction other one is dense reconstruction and how it is done is defined in next sections. for now let's talk about the sparse one here. Stereo geometry estimation is crucial for understanding the relative position and orientation between the two cameras. This is done using the essential matrix E , which encodes the geometric relationship between corresponding points in the two images.

The function `cv2.findEssentialMat()` is used to compute the essential matrix, and `cv2.recoverPose()` is used to recover the relative rotation (R) and translation (T) from the essential matrix.

The code for stereo geometry estimation is as follows:

```
# Find the essential matrix
E, mask = cv2.findEssentialMat(points1, points2, camera_matrix1)

# Recover pose (rotation and translation)
_, R_est, T_est, mask = cv2.recoverPose(E, points1, points2, camera_matrix1)
```

The outputs are:

- `R_est` - Estimated rotation matrix.
- `T_est` - Estimated translation vector.

These values are critical for aligning the two stereo images and are used in the image rectification step.

2.5 SIFT Feature matching

As discussed we tried both sparse and dense reconstruction but our final result is better with dense reconstruction which is discussed in next sections. so for sparse reconstruction we did draw epipolar lines and did feature matching whose results are as follows and we used SIFT for that.

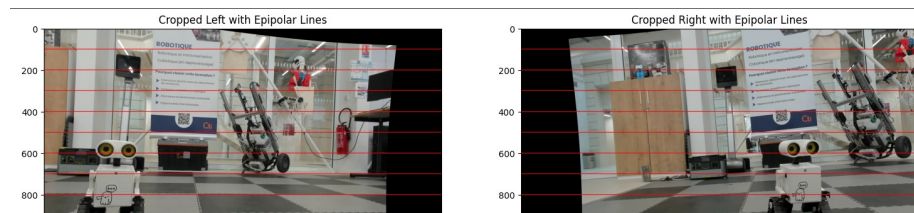


Figure 6: Epipolar lines.

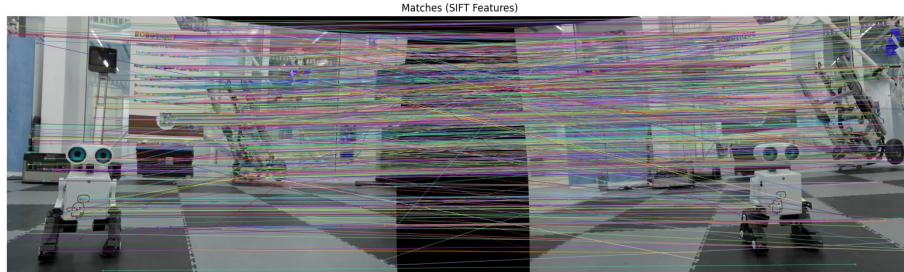


Figure 7: SIFT

2.6 Image Rectification

Image rectification is a crucial step in the stereo vision pipeline that aligns two images from different viewpoints, so that corresponding points in the images lie on the same horizontal line (epipolar alignment). This process is essential for accurate disparity map generation because it simplifies the problem of finding corresponding points between the left and right images. Without rectification, disparity calculations would require searching in a 2D region, making the process more complex.

Rectification is achieved through the following steps:

- Compute the rectification transforms for each camera based on the calibration parameters.
- Apply the transforms to the stereo images using remapping functions.
- Align the stereo images so that corresponding points lie on the same horizontal line, reducing the search space for disparity calculations.

The key function used for image rectification in OpenCV is `cv2.stereoRectify()`, which computes the rectification transforms for the two images based on the camera calibration parameters (intrinsic and extrinsic parameters).

2.6.1 Rectification Process

The rectification process involves the following steps:

1. **Stereo Rectify:** The function `cv2.stereoRectify()` computes the rectification transforms for both the left and right images.
2. **Undistort and Remap:** The function `cv2.initUndistortRectifyMap()` generates the maps for undistorting and rectifying the images. These maps are then applied to the stereo images using `cv2.remap()`.

`cv2.stereoRectify()` computes the following outputs:

- **R1, R2:** Rotation matrices for the left and right cameras, respectively, that describe the transformation required to align the images.

- P1, P2: Projection matrices for the left and right cameras after rectification.
- Q: Disparity-to-depth mapping matrix, used to map the disparity map to 3D coordinates.
- roi1, roi2: Regions of interest for the rectified images.

The code for performing image rectification is as follows:

```
# Perform stereo rectification
R1, R2, P1, P2, Q, roi1, roi2 = cv2.stereoRectify(
    camera_matrix1, dist_coeffs1, camera_matrix2, dist_coeffs2,
    image_size, R_est, T_est)

# Generate the remapping maps
map1_x, map1_y = cv2.initUndistortRectifyMap(
    camera_matrix1, dist_coeffs1, R1, P1, image_size, cv2.CV_32F)
map2_x, map2_y = cv2.initUndistortRectifyMap(
    camera_matrix2, dist_coeffs2, R2, P2, image_size, cv2.CV_32F)

# Remap the original images using the generated maps
left_rectified = cv2.remap(left_image, map1_x, map1_y, interpolation=cv2.INTER_LINEAR)
right_rectified = cv2.remap(right_image, map2_x, map2_y, interpolation=cv2.INTER_LINEAR)

# Display the rectified images
cv2.imshow("Rectified Left", left_rectified)
cv2.imshow("Rectified Right", right_rectified)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

2.6.2 Explanation of Code

- `cv2.stereoRectify()`: This function computes the rectification transforms. It uses the rotation matrix (`R_est`) and translation vector (`T_est`) from the previous stereo geometry estimation step to compute the rotation and projection matrices for both the left and right cameras. - `cv2.initUndistortRectifyMap()`: This function generates the remapping maps (`map1_x`, `map1_y`, `map2_x`, and `map2_y`) that are used to undistort and rectify the original images. The maps are then applied to the left and right images using `cv2.remap()`. - `cv2.remap()`: This function applies the generated remapping maps to the stereo images to obtain the rectified versions of the left and right images.

2.6.3 Rectification Results

The result of the rectification process is two images in which corresponding points are now aligned horizontally, making it easier to compute disparity.

Ideally, there should be minimal vertical disparity, meaning the corresponding points in the images should lie on the same horizontal line. The rectified images are now ready for disparity map generation, as the search space for corresponding points has been reduced to a single row in both images.

Below is an example of the rectified left image after the stereo rectification process:

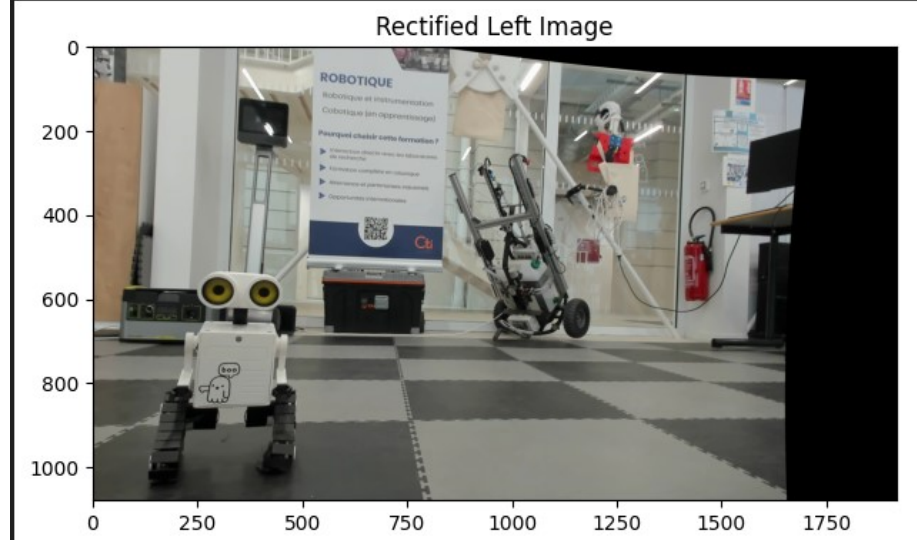


Figure 8: Rectified Left Image After Stereo Rectification.

Similarly, the rectified right image will align its corresponding points along the same horizontal lines, making disparity map generation more efficient and accurate.

2.6.4 Disparity Map Generation

After image rectification, we move to the next step in the stereo vision pipeline: disparity map generation. The rectified images make it easier to match corresponding points between the left and right images, leading to a more accurate disparity map. This process will be discussed in the next section.

2.7 Disparity Map Generation

Disparity map generation is the process of computing the depth information from two rectified stereo images. The disparity map encodes the difference in the position of corresponding points between the left and right images, which is inversely proportional to the depth of the points in the scene. In other words, the larger the disparity, the closer the object is to the cameras.

Stereo vision algorithms typically use block matching techniques to compute the disparity map. Block matching compares corresponding blocks of pixels in the left and right images and calculates the disparity based on the best match between the blocks. The disparity map is then refined to reduce noise and enhance accuracy.

The steps involved in disparity map generation are as follows:

1. **Block Matching:** This method calculates disparity by comparing image patches (blocks) between the left and right rectified images.
2. **WLS Filtering:** The disparity map is filtered using the Weighted Least Squares (WLS) filter to smooth the disparity values while preserving the edges in the image.

The block matching algorithm is implemented using the OpenCV function `cv2.StereoBM.create()`, which generates a disparity map using block matching. The WLS filter, implemented using `cv2.ximgproc.createDisparityWLSFilter()`, is used to refine the disparity map.

2.7.1 Block Matching

The block matching method compares small blocks of pixels in the left and right rectified images and computes the disparity based on the pixel intensity differences between the blocks. The `cv2.StereoBM.create()` function is used to create a block matching object, and the function `compute()` is used to generate the disparity map.

The code for block matching is as follows:

```
# Create StereoBM object for block matching
stereo_bm = cv2.StereoBM_create(numDisparities=16, blockSize=15)

# Compute the disparity map using block matching
disparity = stereo_bm.compute(left_rectified, right_rectified)

# Normalize the disparity map for visualization
disparity_normalized = cv2.normalize(disparity, disparity, 0, 255, cv2.NORM_MINMAX)
disparity_normalized = np.uint8(disparity_normalized)
```

In this code: - `numDisparities`: Specifies the maximum disparity minus one. For example, `numDisparities=16` means that the disparity values can range from 0 to 16. - `blockSize`: Specifies the size of the block (in pixels) used for matching. Larger blocks typically yield smoother disparity maps but may result in less accuracy for fine details.

2.7.2 WLS Filtering

The block matching algorithm can sometimes produce noisy disparity maps, especially in regions with low texture. To refine the disparity map, a Weighted

Least Squares (WLS) filter is applied. The WLS filter smooths the disparity values while preserving the edges of objects in the scene.

The WLS filter is created using `cv2.ximgproc.createDisparityWLSFilter()` and is applied to the disparity map using the `filter()` method.

The code for applying the WLS filter is as follows:

```
# Create a WLS filter object
wls_filter = cv2.ximgproc.createDisparityWLSFilter(stereo_bm)

# Apply the WLS filter to the disparity map
filtered_disparity = wls_filter.filter(disparity, left_rectified)
```

The WLS filter reduces noise and improves the accuracy of the disparity map by taking into account the spatial coherence of the disparity values. This step is important for generating a clean and accurate 3D reconstruction.

2.7.3 Disparity Map Visualization

After generating the disparity map, it is visualized for inspection. The disparity map is normalized to the range of $[0, 255]$ for easier visualization. The disparity map is then displayed using OpenCV's `cv2.imshow()` function.

The code for visualizing the disparity map is as follows:

```
# Display the original disparity map
cv2.imshow('Disparity Map', disparity_normalized)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

The disparity map is shown as an image where the intensity of each pixel represents the disparity at that point. Brighter areas correspond to closer objects, and darker areas correspond to objects farther away from the cameras.

2.7.4 Disparity Map Results

The disparity map obtained after applying block matching and WLS filtering represents the depth information for each pixel in the rectified images. The disparity values are inversely proportional to the depth: higher disparity values correspond to closer objects, while lower disparity values correspond to objects farther away.

Below is an example of the disparity map generated from the rectified stereo images:

The disparity map provides essential depth information that is used in the next stage of the pipeline for 3D reconstruction.

2.8 Next Steps

Once the disparity map is generated, the next step in the stereo vision pipeline is to reconstruct the 3D coordinates of the points in the scene using triangulation, which will be discussed in the following section.

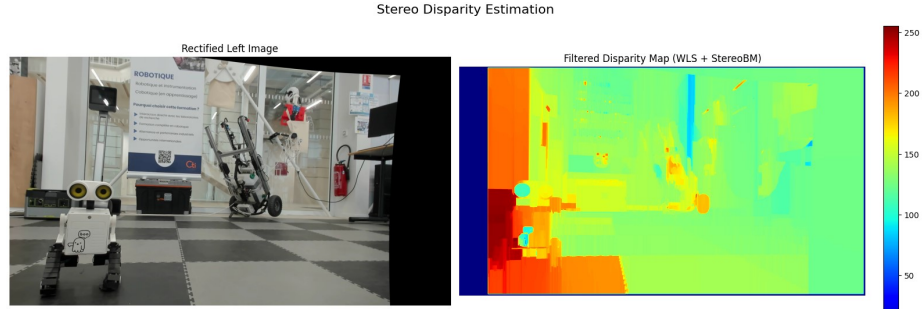


Figure 9: Disparity Map Generated Using Block Matching and WLS Filtering.

2.9 Triangulation & 3D Reconstruction

Once the disparity map is generated, the next step in the stereo vision pipeline is triangulation. Triangulation is the process of determining the 3D coordinates of points in the scene by using the disparity values obtained from the rectified stereo images. The disparity map provides the horizontal shift between corresponding points in the left and right images, which is inversely proportional to the depth of the points in the scene. By using the camera matrices and the disparity values, we can compute the 3D coordinates of each point.

The general process of triangulation involves:

- Using the disparity map to estimate the depth of each pixel in the rectified images.
- Reprojecting the 2D image coordinates into 3D space using the camera matrices and the disparity values.
- Constructing a 3D point cloud that represents the scene.

The disparity map provides the horizontal shift between corresponding points, which can be converted into depth using the equation:

$$Z = \frac{f \cdot B}{d}$$

where:

- Z is the depth (distance from the camera to the point),
- f is the focal length of the camera,
- B is the baseline (distance between the two cameras),
- d is the disparity value for the corresponding point.

Using this equation, we can compute the depth for each pixel and use it to reproject the 2D points into 3D space.

2.9.1 Reprojection of Image Points to 3D Coordinates

To convert the disparity values into 3D coordinates, the function `cv2.reprojectImageTo3D()` is used. This function takes the disparity map and the disparity-to-depth mapping matrix (Q) computed earlier during stereo rectification, and it returns the 3D coordinates for each point.

The code for triangulation and 3D reconstruction is as follows:

```
# Reproject the disparity map to 3D coordinates using the Q matrix
points_3D = cv2.reprojectImageTo3D(disparity, Q)

# Filter out points where disparity is 0 (invalid points)
valid_points = points_3D[disparity > 0]

# Get the colors of the points from the left rectified image
colors = cv2.cvtColor(left_rectified, cv2.COLOR_BGR2RGB)

# Visualize the 3D point cloud using matplotlib
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
ax.scatter(valid_points[:, 0], valid_points[:, 1], valid_points[:, 2], c=colors)
plt.show()
```

In this code: - `**cv2.reprojectImageTo3D()`: This function takes the disparity map and the disparity-to-depth mapping matrix Q , which is computed during stereo rectification. It reprojects the 2D pixel coordinates into 3D space, creating a 3D point cloud. - `**Filtering Invalid Points`: Points with a disparity of 0 (which represent no match between the left and right images) are removed from the point cloud. - `**Color Mapping`: The colors of the points are taken from the left rectified image and are used to visualize the 3D point cloud.

2.9.2 3D Point Cloud Visualization

The 3D points are visualized using a 3D scatter plot in matplotlib. Each point in the point cloud is represented as a point in 3D space, and its color is determined by the corresponding pixel's color in the left rectified image.

The code for visualizing the 3D point cloud is as follows:

```
# Create a 3D scatter plot to visualize the point cloud
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')

# Plot the 3D points
ax.scatter(valid_points[:, 0], valid_points[:, 1], valid_points[:, 2], c=colors)
ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_zlabel('Z')
plt.show()
```

This code generates a 3D scatter plot where each point represents a point in the reconstructed 3D scene. The points are colored based on the corresponding pixel color from the left image, giving a realistic view of the scene.

2.9.3 3D Reconstruction Results

The result of this step is a dense 3D point cloud that represents the spatial arrangement of the objects in the scene. This point cloud can be further processed for 3D modeling, object recognition, or other applications requiring depth information. The following image shows an example of the 3D point cloud generated by the system:

Interactive 3D Point Cloud

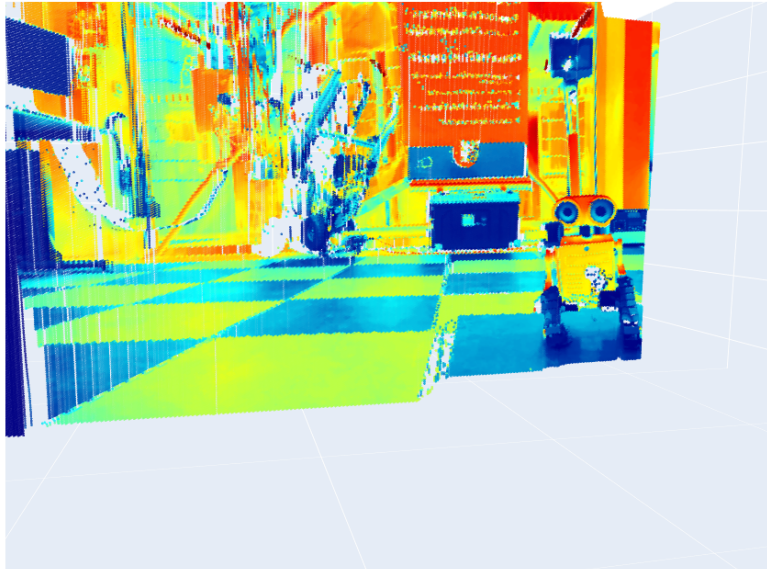


Figure 10: 3D Point Cloud Generated from the Disparity Map.

The point cloud shows the relative positions

3 Discussion

The stereo vision pipeline developed in this project successfully generates dense 3D point clouds from two synchronized stereo images captured by standard webcams. The system demonstrates the feasibility of using low-cost hardware

to achieve accurate 3D reconstructions, making it accessible for a wide range of applications, including robotics, object detection, and augmented reality. However, several factors impact the overall performance of the system, including the accuracy of calibration, lighting conditions, and the computational demands of real-time processing.

3.1 Performance Evaluation

The system performs well in generating 3D reconstructions from stereo images. Key factors influencing the performance include:

- **Camera Calibration:** Accurate camera calibration is critical for the success of the stereo vision pipeline. The use of a checkerboard pattern for calibration, along with precise corner detection, ensures that the intrinsic and extrinsic parameters are well-defined. However, any error in the calibration process could lead to inaccuracies in the depth map and 3D point cloud.
- **Disparity Map Generation:** The block matching algorithm used for disparity map generation provides satisfactory results in textured areas. However, in regions with little texture (such as flat, uniform surfaces), the disparity map may suffer from noise, which affects the quality of the 3D reconstruction. The use of the WLS filter improves the disparity map, but there are still limitations in very low-texture areas.
- **Real-Time Processing:** The system is able to generate 3D point clouds in real-time, provided the images are of moderate resolution and the hardware used is sufficient for processing. The computational demand of real-time disparity map generation and triangulation is a factor that needs to be optimized, especially for higher-resolution images or more complex scenes.

Despite these challenges, the system produces reliable 3D point clouds that accurately represent the spatial relationships in the scene. The final 3D point cloud can be used for a variety of applications, including navigation for robots, virtual reality, and scene reconstruction.

3.2 Challenges and Limitations

While the system is functional, there are several challenges and limitations that need to be addressed:

- **Calibration Accuracy:** The accuracy of the calibration process is critical. Errors in corner detection or incorrect camera placement can lead to poor calibration, which ultimately affects the quality of the 3D reconstruction. To mitigate this, it is essential to ensure that the calibration images are captured in different positions and orientations, and that the checkerboard pattern is well-lit and clearly visible.

- **Low-Texture Areas:** The block matching algorithm performs well in areas with distinct textures. However, in areas with low texture (e.g., plain walls or smooth surfaces), the disparity map can become noisy, leading to incorrect depth estimates. This is a common issue in stereo vision systems and can be mitigated by using more advanced algorithms, such as semi-global matching or deep learning-based approaches for disparity estimation.
- **Lighting Conditions:** Stereo vision systems are sensitive to lighting conditions. Uneven lighting can cause shadows, reflections, or overexposure, which can negatively affect the quality of the disparity map. Consistent lighting is essential for achieving accurate results, and the system’s performance may degrade under poor lighting conditions.
- **Real-Time Performance:** Although the system is designed to work in real-time, the computational demands of generating disparity maps and performing triangulation can be significant, particularly when working with high-resolution images. The processing time increases as the resolution and complexity of the scene increase, which may affect the system’s ability to operate in real-time for large or complex environments.

3.3 Potential Improvements

Several improvements can be made to enhance the accuracy and performance of the stereo vision pipeline:

- **Use of Advanced Disparity Algorithms:** While the block matching algorithm is suitable for many applications, more advanced algorithms, such as semi-global matching (SGM) or deep learning-based disparity estimation, could improve the quality of the disparity map, especially in low-texture regions. These algorithms are more robust and provide better results in challenging scenarios.
- **Better Camera Calibration Methods:** To further improve the accuracy of the 3D reconstruction, more sophisticated calibration techniques could be employed. Using multiple calibration patterns or including dynamic calibration during the reconstruction process could help refine the camera parameters and correct any misalignments.
- **Improved Lighting and Camera Setup:** To address issues with lighting, using controlled lighting environments or more advanced camera setups, such as using cameras with better low-light performance or adding infrared (IR) cameras, could help improve depth estimation under varying lighting conditions.
- **Optimizing Real-Time Processing:** To optimize real-time performance, hardware acceleration (e.g., using GPUs for parallel processing) could be

employed. Additionally, techniques such as downsampling the input images or optimizing the disparity map generation algorithm could help reduce the computational load and enable real-time operation for higher-resolution images.

- **Surface Reconstruction and Mesh Generation:** Once the 3D point cloud is generated, surface reconstruction techniques, such as Poisson Surface Reconstruction or mesh generation algorithms, could be applied to convert the point cloud into a solid mesh, which would provide more detailed and usable 3D models.

3.4 Applications

The 3D point cloud generated by this stereo vision system can be used in various fields:

- **Robotics:** In autonomous navigation, robots can use the 3D point cloud to detect obstacles, plan paths, and understand the environment in three dimensions.
- **Object Detection:** The depth information in the 3D point cloud can be used for detecting and tracking objects in the scene, which is useful in industrial applications, security, and surveillance.
- **Augmented Reality:** The 3D reconstructions can be used to create realistic virtual objects that interact with the physical world in augmented reality applications.
- **3D Mapping and Modeling:** The point cloud can be used to generate 3D models of real-world environments for virtual simulations, architectural design, or cultural heritage preservation.

4 Conclusion

This project successfully demonstrated the feasibility of generating dense 3D reconstructions using stereo vision techniques with low-cost hardware. By leveraging two parallel webcams and OpenCV, the system is able to compute disparity maps and generate 3D point clouds in real-time. This low-cost setup makes it accessible for various applications, from educational purposes to practical implementations in robotics, augmented reality, and 3D modeling.

The key objectives of the project were achieved, including accurate camera calibration, stereo geometry estimation, image rectification, and disparity map generation. The 3D point cloud generated by the system provides a reliable representation of the spatial arrangement of objects in the scene. While the system performs well in controlled environments, several challenges were encountered, including calibration accuracy, the handling of low-texture areas, and the computational demands of real-time processing. These challenges are common in

stereo vision systems and can be addressed with improvements in both software and hardware.

The system’s performance was evaluated based on the quality of the 3D point cloud and the real-time processing capability. The results demonstrated that the system can generate accurate depth maps and 3D reconstructions under typical conditions. However, for more complex environments or high-resolution images, the system’s performance may need to be optimized further.

4.1 Impact of the Work

This project contributes to the field of stereo vision by providing an accessible and low-cost method for 3D reconstruction. By utilizing widely available webcams and open-source software, the system offers an affordable alternative to expensive stereo camera rigs, making it suitable for educational purposes, research, and prototyping. The results of this work can be applied in various domains, including robotics, where depth perception is crucial for navigation and obstacle detection, and in augmented reality, where accurate 3D models are needed for realistic virtual interactions.

Furthermore, the system provides a foundation for further research and development in the field of stereo vision. The techniques developed here can be extended to more complex setups, such as multi-camera systems or real-time processing with advanced algorithms, to improve the accuracy and robustness of 3D reconstructions.

4.2 Future Work

There are several avenues for improving and extending the work presented in this project:

- **Advanced Disparity Algorithms:** While the block matching algorithm works well in many cases, more advanced algorithms, such as semi-global matching (SGM) or deep learning-based methods, could provide better results, especially in low-texture areas. These algorithms could improve the accuracy of the disparity map and the 3D reconstruction.
- **Camera Setup Enhancements:** Using cameras with higher resolution, improved low-light performance, or even infrared (IR) cameras could enhance the quality of the depth maps. Additionally, a more controlled lighting environment could help reduce noise and improve accuracy.
- **Real-Time Processing Optimization:** To enable real-time operation for larger scenes or higher-resolution images, hardware acceleration (such as GPU processing) could be utilized. Furthermore, optimizing the disparity map generation and triangulation algorithms could reduce computational demands.
- **Surface Reconstruction:** After generating the 3D point cloud, surface reconstruction techniques could be applied to convert the point cloud into

a 3D mesh. This would provide a more complete and usable 3D model for applications such as 3D printing, simulation, or virtual reality.

- **Integration with Other Sensors:** Integrating the stereo vision system with other sensors, such as LiDAR or depth cameras, could improve the system's robustness and accuracy, especially in challenging environments or for large-scale 3D reconstructions.

In conclusion, this project successfully demonstrates the power of stereo vision for 3D reconstruction using low-cost hardware. With further optimization and enhancements, this system has the potential to be applied in a wide range of fields, providing an accessible solution for generating 3D models from real-world environments.